Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to implement a Multi-Layer Perceptron using TensorFlow/Keras for multi-class image classification on the Fashion-MNIST dataset. This covers the full pipeline – loading and preprocessing image data, flattening it into a form a Dense network can consume, building and training the MLP, evaluating it with standard classification metrics, and finally performing automated hyperparameter optimization to find a better-performing configuration than the baseline.

2. Background Theory

A Multi-Layer Perceptron is a feedforward neural network made up of an input layer, one or more hidden layers, and an output layer. Each layer applies a linear transformation followed by a non-linear activation function:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f(z^{(l)})$$

where $W^{(l)}$ and $b^{(l)}$ are the weights and bias of layer l, and f is the activation function (ReLU in the hidden layers of this experiment). Stacking layers this way lets the network build up increasingly abstract representations of the input as it moves deeper.

For a multi-class classification problem such as this one (10 clothing categories), the output layer uses the Softmax activation, which converts the raw output scores into a probability distribution over the classes:

$$\hat{y} = \text{Softmax}(z)$$

The network is trained by minimizing the Categorical Cross Entropy loss between the predicted probability distribution and the true one-hot encoded label:

$$L = -\sum_i y_i \log(\hat{y}_i)$$

The architecture used in this experiment is the one recommended in the manual:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset

Dataset: Fashion-MNIST

Training Images	60,000
Testing Images	10,000
Classes	10
Image Size	28×28

The dataset was loaded directly from `tensorflow.keras.datasets.fashion_mnist`, which downloads and returns the data already split into training and testing sets. Every image is a single-channel 28×28 grayscale image belonging to one of ten clothing categories – T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot.

Since Dense layers expect a one-dimensional feature vector rather than a 2D image, each 28×28 image needs to be flattened into a 784-length vector before it can be fed into the network:

$$28 \times 28 = 784$$

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

4. Experimental Procedure

Task 1: Dataset Exploration. I loaded the dataset, printed the shapes of the train and test splits, displayed ten sample images with their class labels, and plotted the class distribution across the training set to check whether the classes were balanced.
Task 2: Data Preprocessing. Each image was flattened from 28×28 down to a 784-length vector, and pixel values were rescaled from $[0, 255]$ to $[0, 1]$ by dividing by 255. The integer class labels were then one-hot encoded using `to_categorical` so they could be used with categorical cross-entropy.
Task 3: Model Construction. I built the MLP as a Keras `Sequential` model using the architecture given in the manual: an input of 784 features, a Dense(128, ReLU) layer, a Dense(64, ReLU) layer, and a Dense(10, Softmax) output layer.
Task 4: Model Training. I compiled the model with the Adam optimizer, categorical cross-entropy loss, and accuracy as the metric to track, then trained it for 20 epochs with a batch size of 32. 10% of the training data was held back for validation.
Task 5: Model Evaluation. The trained model was evaluated on the 10,000-image test set, reporting accuracy, precision, recall, F1-score, a confusion matrix, and a full classification report.

5. Source Code

The complete source code for this experiment – dataset exploration, preprocessing, model construction, training, evaluation, and hyperparameter optimization – is available on GitHub at:

`https://github.com/Lunarya-git/Deep-Learning-Experiments/tree/main/Experiment_2_multi-layer-perceptron`

6. Hyperparameter Optimization

Instead of picking hyperparameters by hand, an automated search was run using `RandomizedSearchCV` together with the SciKeras wrapper (`KerasClassifier`). A model-builder function that takes the number of hidden layers, neurons per layer, learning rate, optimizer, activation function, and dropout rate as

arguments and returns the corresponding compiled MLP was implemented. This function was wrapped with SciKeras and handed to `RandomizedSearchCV` along with the full search space given in the manual:

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate	0.0, 0.2, 0.5

Running RandomizedSearchCV with 5 folds is expensive to run for 60,000 training images on Google Colab and hence the search was run on a stratified 10,00 image sample. After the configuration was decided, the new model was retrained on the complete 60,000 dataset. Search took 1633.11 seconds (27 minutes approx) and finally got an configuration with mean 5-fold cross-validation accuracy of 0.8600. This configuration was used to build a fresh model which was then trained on the entire training set and then evaluated on the test set. Then this was compared fairly against baseline model from Task 4.

7. Results

Best Hyperparameters

Hidden Layers	2
Hidden Neurons	256
Learning Rate	0.001
Batch Size	32
Optimizer	Adam
Activation Function	Sigmoid
Epochs	30
Dropout	0.0
Cross-validation Accuracy	0.8600
Testing Accuracy	0.8910

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8851	0.8910
Precision	0.8852	0.8933
Recall	0.8851	0.8910
F1-score	0.8846	0.8912
Training Time (s)	113.46	151.39

The baseline model (Task 3/4 architecture, 20 epochs, batch size 32) finished with a training accuracy of 93.31% and a test accuracy of 88.51%. Looking at the classification report, it did best on visually

distinctive classes like Trouser (F1 = 0.98), Sandal (F1 = 0.96), and Bag (F1 = 0.96), and was clearly weakest on Shirt (precision 0.74, recall 0.65, F1 = 0.69) – which makes sense given how similar Shirt looks to T-shirt/top, Pullover, and Coat.

The optimized model (2 hidden layers of 256 neurons, Sigmoid activation, Adam at learning rate 0.001, batch size 32, 30 epochs, no dropout) pushed test accuracy up to 89.10%, about 0.6 percentage points over the baseline, but it cost roughly 34% more training time (151.39s vs 113.46s). So the gain is real, just not large – the extra width (256 vs 128/64 neurons) and the longer training schedule helped a bit, but the search never landed on anything dramatically better than the manual's baseline architecture.

8. Plots with Inference

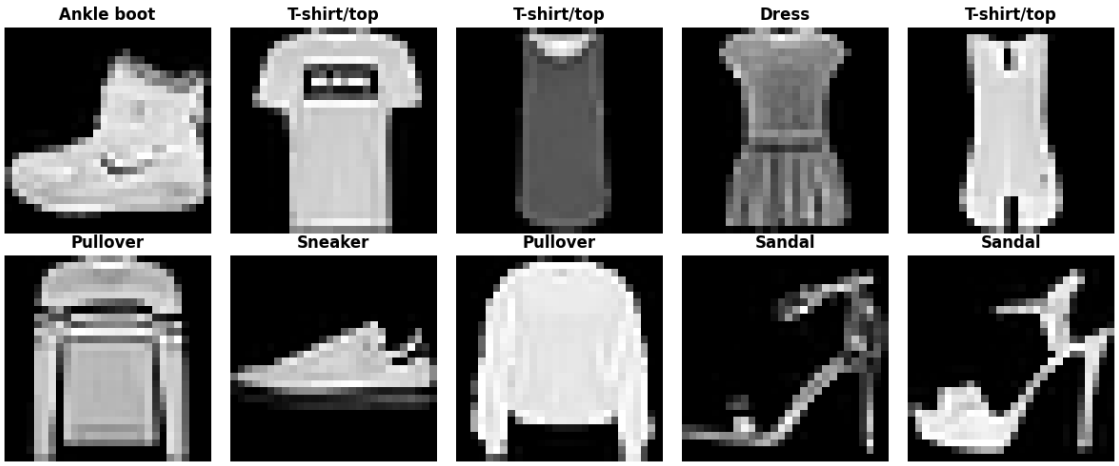

Figure 1: Ten sample images from the training set. These confirm the images are exactly what's expected – low-resolution (28×28) grayscale clothing photos, correctly labelled. A few categories (tops and outerwear in particular) already look similar to the naked eye, which is a hint of the confusion the model runs into later.

Figure 2: Class distribution in the training set. All ten classes have exactly 6,000 images each, so Fashion-MNIST is perfectly balanced. That makes accuracy a reasonable headline metric to use here, and there was no need for class-weighting or resampling during training.

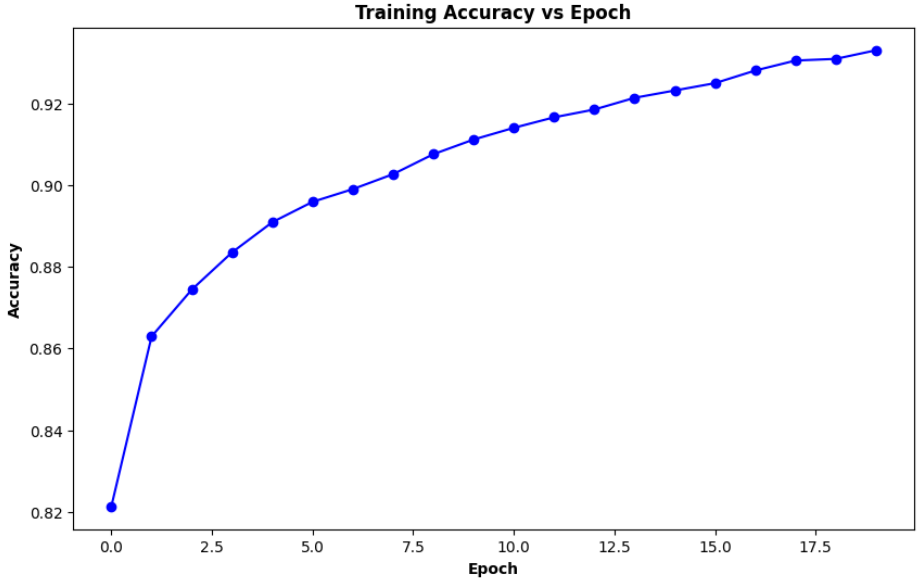

Figure 3: Training accuracy vs epoch (baseline model). Accuracy climbs steeply at first (82.1% up to about 89% by epoch 5), then keeps rising more slowly to 93.3% by epoch 20. The curve is smooth and monotonic, which makes sense since this is the same data the model is being optimized on.

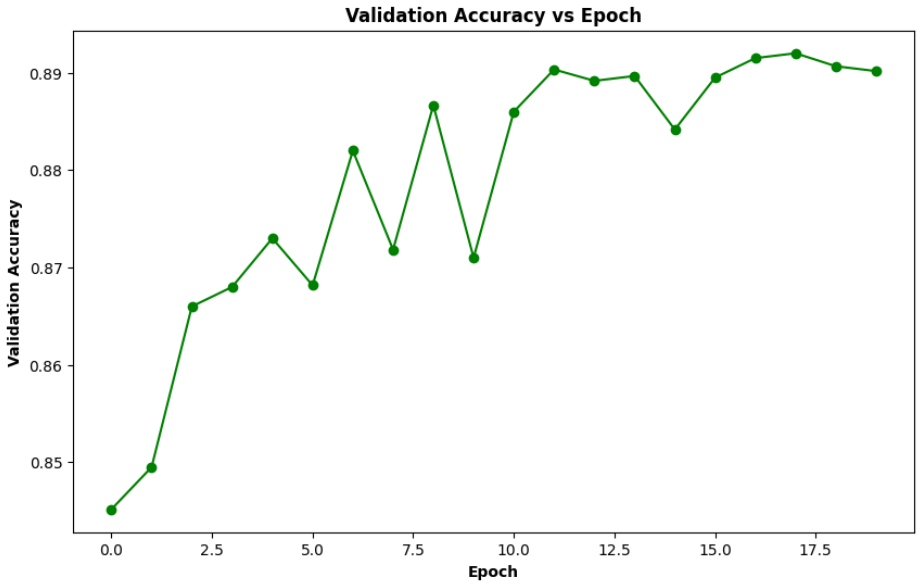

Figure 4: Validation accuracy vs epoch (baseline model). Validation accuracy trends upward too (84.5% to about 89%), but it's noticeably noisier than training accuracy, with a dip and recovery around epoch 10. That kind of fluctuation is normal for a held-out set. It also plateaus earlier, around epoch 9, well before the training curve does.

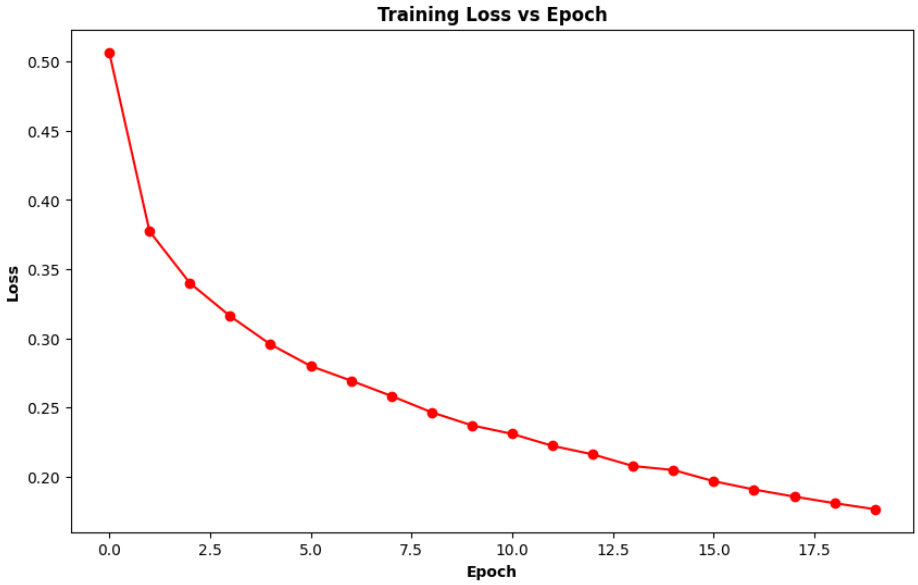

Figure 5: Training loss vs epoch (baseline model). Loss falls smoothly from 0.507 down to 0.176 across the 20 epochs, mirroring the training accuracy curve. Nothing unstable happening in the optimization here.

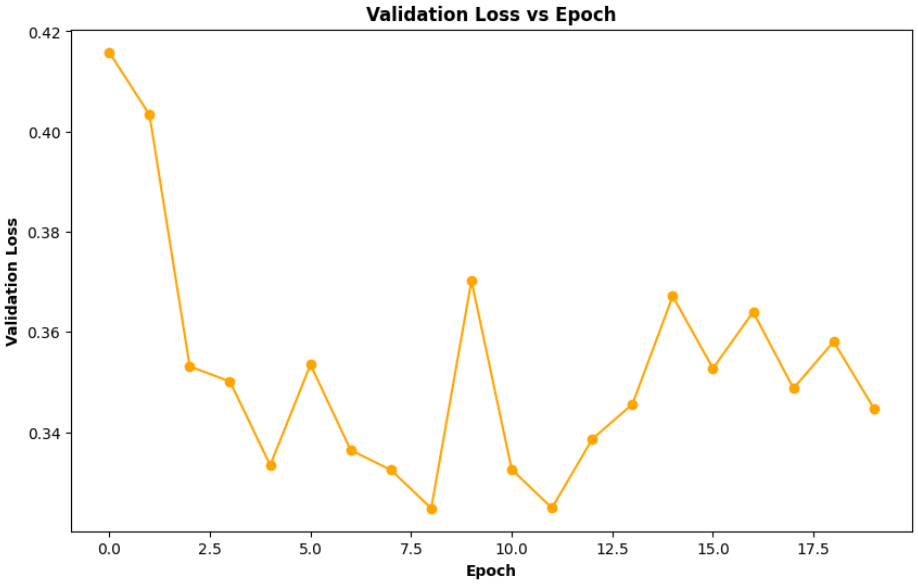

Figure 6: Validation loss vs epoch (baseline model). Validation loss drops until around epoch 8–9 (down to roughly 0.325), then starts creeping back up even as training loss keeps falling. That gap is a fairly classic overfitting signal, though it's worth noting validation accuracy itself stays roughly flat through this period.

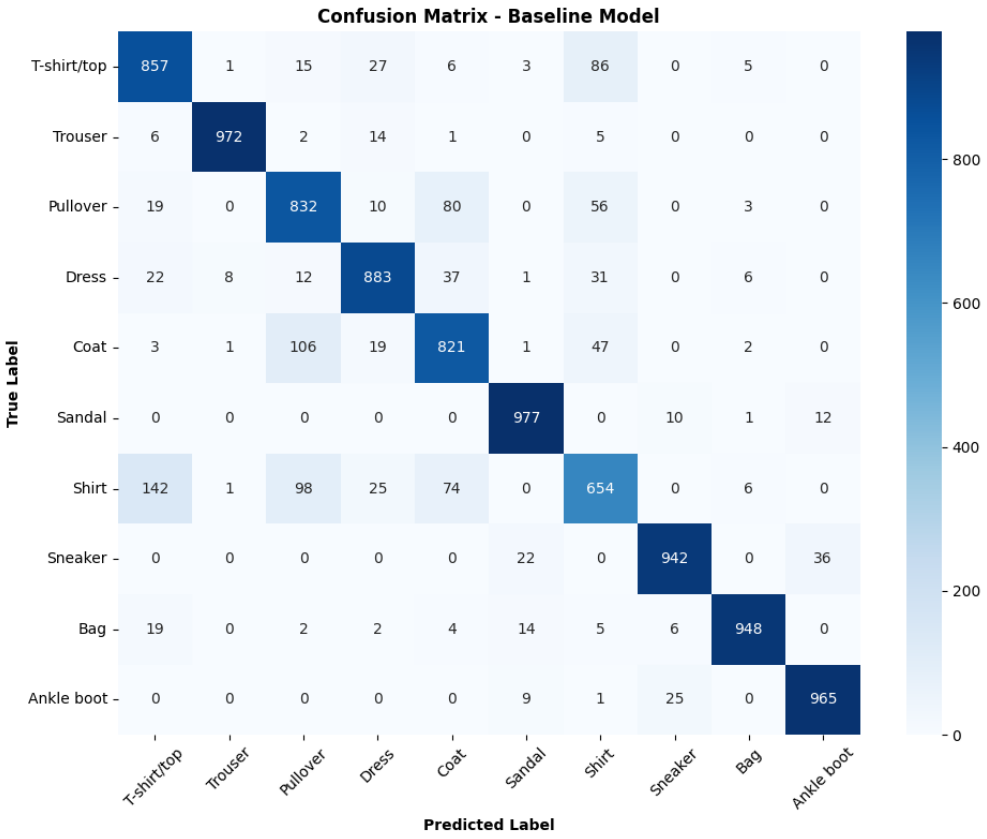

Figure 7: Confusion matrix on the test set (baseline model). The diagonal is strong almost everywhere, but there's a clear block of confusion between Shirt, T-shirt/top, Pullover, and Coat – the same visually similar upper-body garments flagged in Figure 1, and the reason Shirt came out as the weakest class in the classification report.

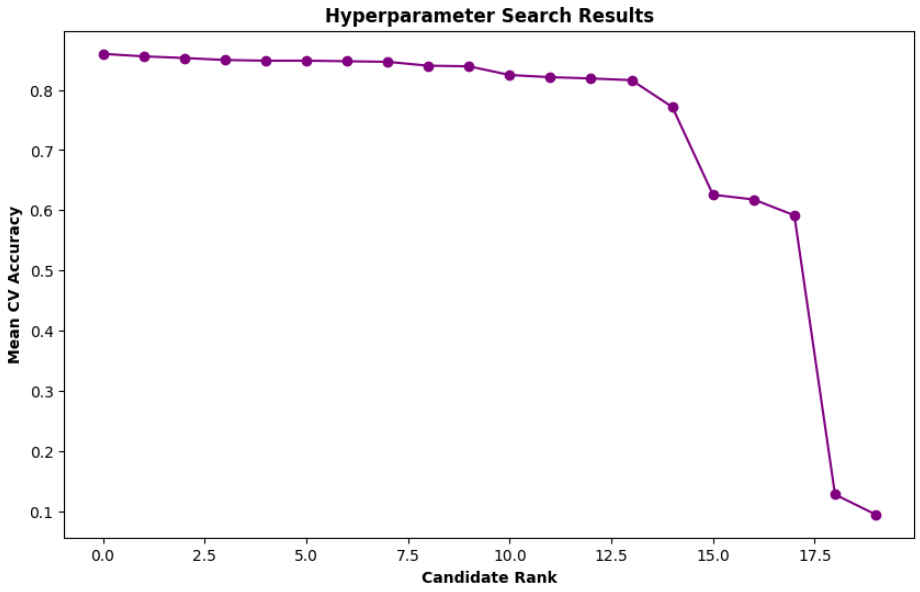

Figure 8: Hyperparameter search results. Candidates are ranked by mean cross-validation accuracy, best to worst. The score drops off fairly quickly after the top few, so only a handful of the 20 sampled combinations got close to the best score of 0.86 – most of the rest trailed noticeably behind.

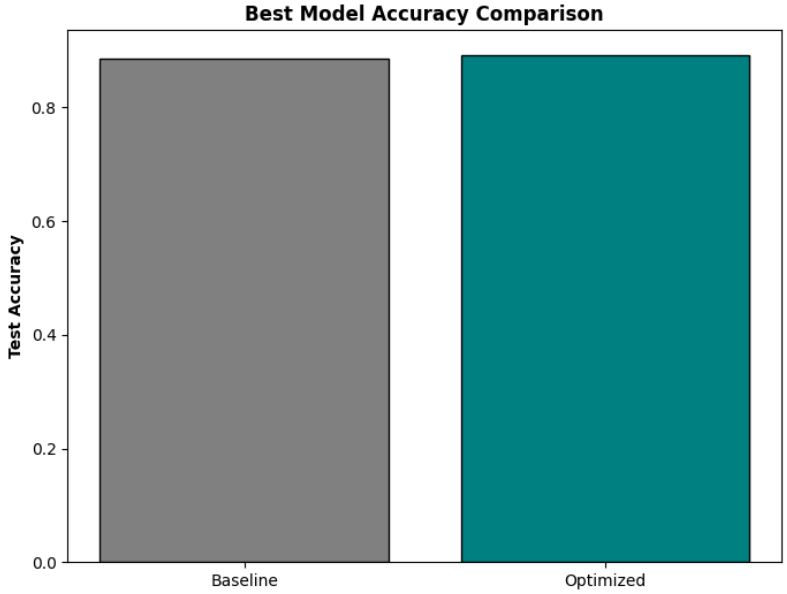

Figure 9: Best model accuracy comparison. The optimized model's test accuracy (89.10%) is only a little higher than the baseline's (88.51%) – visual confirmation that tuning helped, but only incrementally, for this architecture and dataset.

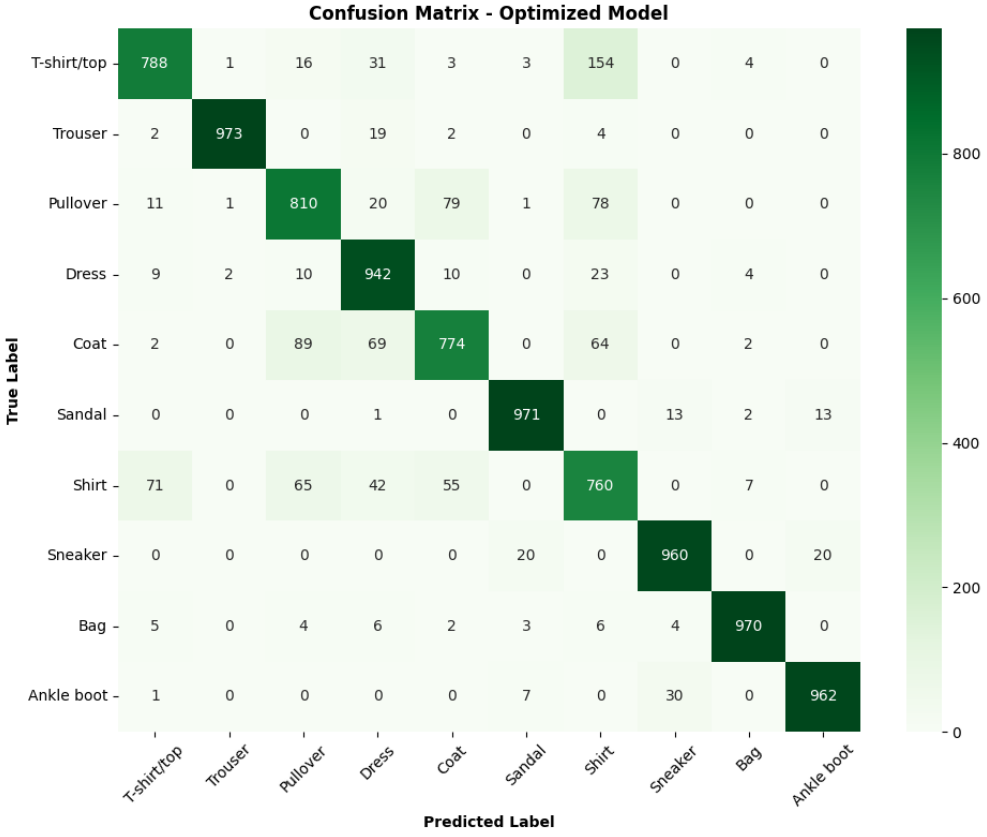

Figure 10 (supplementary): Confusion matrix on the test set (optimized model). The pattern here looks a lot like the baseline's – Shirt is still the hardest class to separate from T-shirt/top, Pullover, and Coat. It seems the hyperparameter search sharpened precision on classes that were already easy more than it fixed this particular confusion.

9. Discussion

1. Which optimization method was used?
RandomizedSearchCV with 5-fold cross-validation, using the SciKeras `KerasClassifier` wrapper, as the manual recommends. To keep it computationally feasible, I ran the search on a 10,000-image stratified subsample of the training data, then retrained the winning configuration on the full 60,000-image training set.

2. Which hyperparameters were selected?
2 hidden layers, 256 neurons per hidden layer, Sigmoid activation, the Adam optimizer at a learning rate of 0.001, batch size 32, 30 training epochs, and no dropout (0.0).

3. Did optimization improve performance?
Yes, but only slightly. Test accuracy went from 88.51% with the baseline to 89.10% after tuning, and precision, recall, and F1 all moved by roughly the same small amount. That gain wasn't free, though: training time went up from about 113 seconds to 151 seconds, close to 34% more, just to pick up 0.6 percentage points of accuracy.

4. Which hyperparameter had the greatest impact?
Looking at the winning configuration and the shape of the search results in Figure 8, the number of neurons and the number of training epochs seem to matter most – the search picked the widest available layer (256 neurons) and the largest epoch budget (30) over smaller, cheaper options. Learning rate looks important too: the search converged on a moderate value (0.001) rather than the largest candidate (0.1), which lines up with the well-known problem of high learning rates destabilizing training in Dense networks with Sigmoid/Softmax layers.

5. Compare Grid Search and Randomized Search.
GridSearchCV tries every combination in the search space, so it's guaranteed to find the best one within the grid, but the cost scales combinatorially. For this search space ($3 \times 4 \times 3 \times 4 \times 3 \times 3 \times 3 \times 3$ combinations, each times 5 folds), a full grid search would mean thousands of model fits – not really practical on limited hardware. RandomizedSearchCV instead samples a fixed number of combinations from that same space (20 here), accepting a small chance of missing the true optimum in exchange for a much smaller computational bill. That trade-off is exactly why the manual recommends it for a search space this large.

6. Recommend the best MLP configuration.
I'd recommend the configuration the search found – 2 hidden layers of 256 neurons, Sigmoid activation, Adam at learning rate 0.001, batch size 32, 30 epochs, no dropout – since it gave the best test performance of everything evaluated. That said, the baseline model's validation loss curve (Figure 6) already showed mild overfitting by epoch 20, and the optimized model trains for even longer with more parameters and zero dropout. So in practice, adding a small amount of dropout (say 0.2) would probably be a sensible tweak if this configuration were pushed further – trained longer, or on less data.

12. K4-Question

Code the perceptron learning algorithm for the XOR gate using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python's built-in packages. Analyze and identify the pattern that prevents the algorithm from converging. (K4)

12.1 Why a Single-Layer Perceptron Cannot Be Used Here

XOR isn't linearly separable: its two classes, $\{(0,0),(1,1)\} \to 0$ and $\{(0,1),(1,0)\} \to 1$, sit on diagonally opposite corners of the unit square, so no single straight line can separate them. A single-layer perceptron can only draw one linear decision boundary ($\mathbf{w}^T\mathbf{x} + b = 0$), which means it's mathematically incapable

of solving XOR no matter how it's trained. For that reason, this question is implemented with a **Multi-Layer Perceptron (MLP)** instead: an input layer of 2 units, one hidden layer of 2 units (Sigmoid activation), and a single Sigmoid output unit – the classic minimal architecture that can represent XOR.

12.2 MLP Architecture and Training Rule

$$2 \text{ Inputs} \rightarrow \text{Dense}(2, \text{Sigmoid}) \rightarrow \text{Dense}(1, \text{Sigmoid})$$

Forward pass:

$$z^{(1)} = \mathbf{x}W_1 + b_1, \qquad a^{(1)} = \sigma(z^{(1)}), \qquad z^{(2)} = a^{(1)}W_2 + b_2, \qquad \hat{y} = \sigma(z^{(2)})$$

Network was trained with full-batch gradient descent (all 4 XOR patterns per update), using the mean squared error loss $L = \frac{1}{4}\sum(y - \hat{y})^2$ and the sigmoid derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, backpropagated through both layers with a learning rate of $\eta = 1.0$ for 6000 epochs. Since training here is gradient-based rather than the single-sample perceptron update rule, one "update" means one full-batch gradient step (epoch); weights were logged every 200 epochs.

12.3 Weight Updates

Table 1: Weight snapshots during MLP training on XOR (random initialization), logged every 200 epochs; 10 representative rows out of 31 logged snapshots over 6000 epochs are shown. W_1 is the 2×2 input→hidden weight matrix, W_2 is the 2×1 hidden→output weight vector.

Epoch	Loss (MSE)	W_1 (input→hidden)	W_2 (hidden→output)
0	0.28630	`[-0.25, 0.90; 0.46, 0.20]`	`[-0.69; -0.69]`
200	0.25008	`[-0.28, 0.84; 0.44, 0.05]`	`[-0.45; -0.35]`
400	0.24999	`[-0.29, 0.83; 0.45, -0.04]`	`[-0.46; -0.33]`
1000	0.24973	`[-0.34, 0.87; 0.53, -0.22]`	`[-0.48; -0.34]`
1600	0.24857	`[-0.53, 1.04; 0.77, -0.45]`	`[-0.57; -0.46]`
2400	0.17861	`[-2.23, 2.52; 2.51, -2.10]`	`[-1.86; -1.77]`
3200	0.02483	`[-4.56, 4.73; 4.75, -4.50]`	`[-4.55; -4.55]`
4000	0.00979	`[-5.16, 5.33; 5.35, -5.12]`	`[-5.53; -5.53]`
5000	0.00524	`[-5.51, 5.68; 5.69, -5.47]`	`[-6.15; -6.15]`
6000	0.00351	`[-5.71, 5.88; 5.90, -5.68]`	`[-6.54; -6.55]`

12.4 Decision Boundary Plots

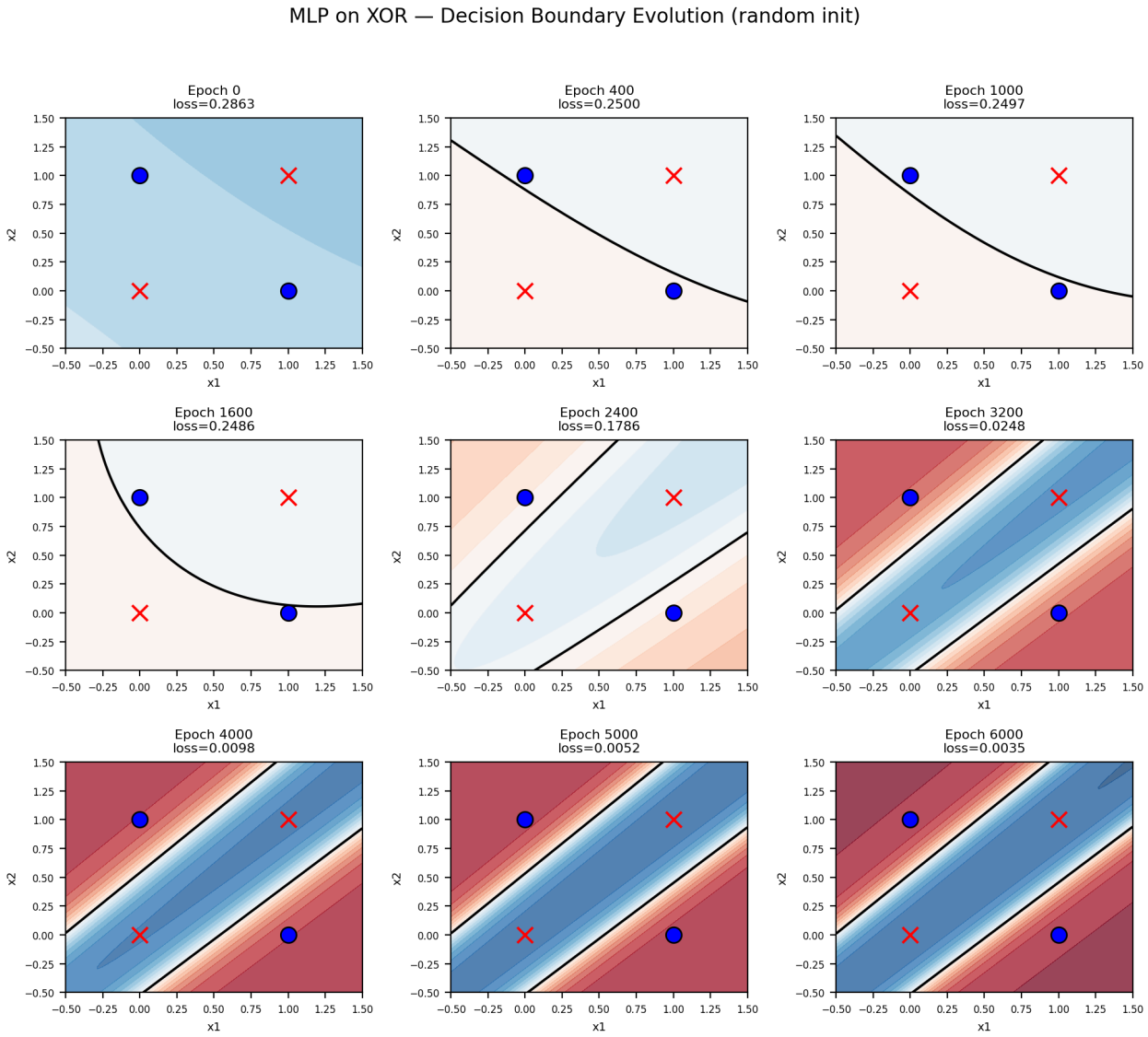

Figure 1: Evolution of the MLP's decision boundary over training. Unlike the single straight line a perceptron is limited to, the MLP's boundary is curved – it bends around to correctly enclose the two diagonal corners belonging to class 1, something no single-layer perceptron could ever do.

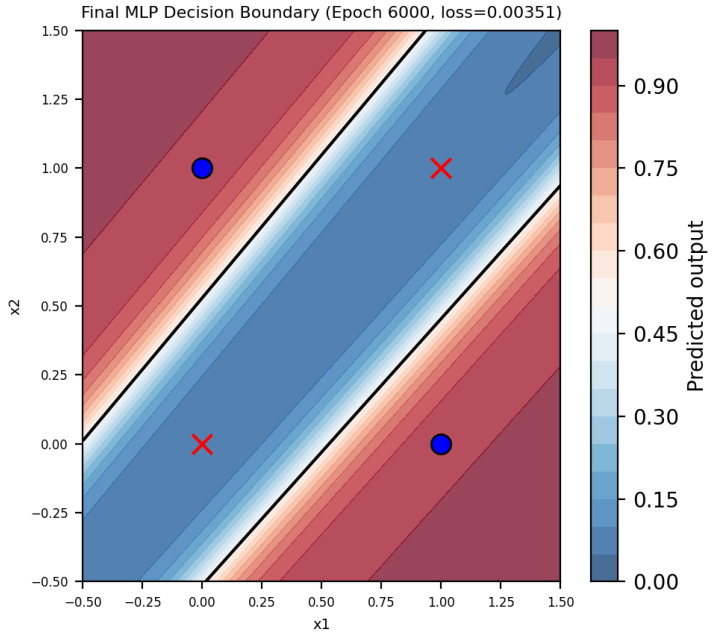

Figure 2: Final decision boundary after 6000 epochs (random initialization). The colour scale shows the network's continuous output; the black contour marks the $\hat{y} = 0.5$ threshold. All four XOR points are now correctly separated.

12.5 Analysis: The Pattern That Prevents Convergence

Training the exact same MLP architecture twice, from two different starting points, shows what actually governs whether it converges:

Initialization	**Final Loss (6000 epochs)**	**Outcome**
All weights = 0	0.25000 (unchanged from epoch 0)	Never converges
Small random weights	0.00351	Converges

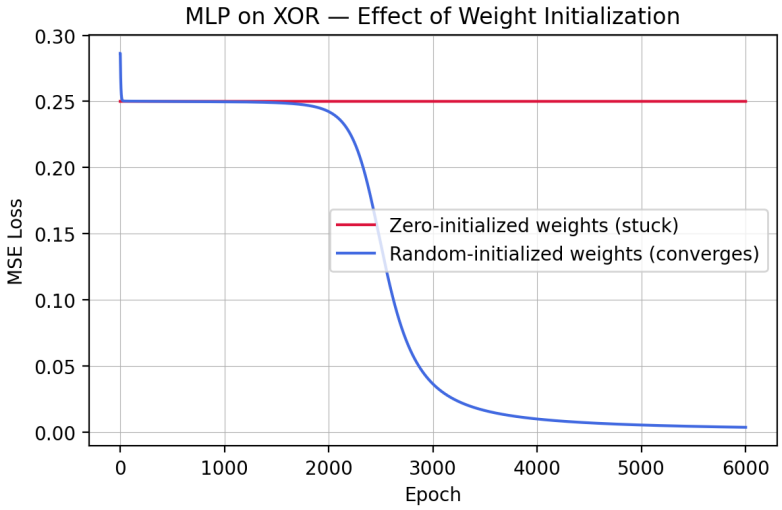

Figure 3: Training loss vs. epoch for zero-initialized weights versus randomly-initialized weights. The zero-init run is perfectly flat at 0.25 for all 6000 epochs; the random-init run steadily decreases to near zero.

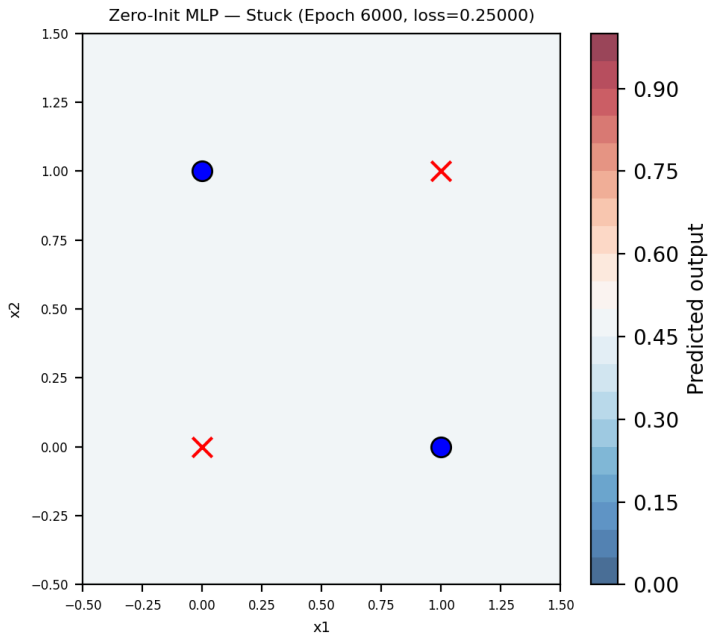

Figure 4: The zero-initialized MLP after 6000 epochs. Both hidden units and both output weights remain *exactly* zero – the network never breaks out of this state and simply outputs 0.5 for every input.

The pattern: symmetric (zero) initialization freezes the network. When every weight starts at zero, both hidden units see identical inputs, compute identical activations, and get identical gradients at every backprop step. Nothing in the update rule makes the two hidden units diverge from each other, so they move in perfect lock-step forever – and since their starting value and gradient are both exactly zero, W_1 and W_2 simply never leave zero (this shows up both in the table in Section 12.3 and in the loss curve above, which stays pinned at 0.25 – exactly the loss of a network that just outputs 0.5 for every input). This is the well-known **symmetry problem**: a hidden layer with identical (or all-zero) weights ends up behaving like a single hidden unit no matter how many units it actually has, because

they can never specialize to pick up different features. It's a different kind of failure from the single-layer perceptron's – that one is a hard geometric limit where no set of weights can ever work, whereas here a solution does exist (the random-init run proves it) – the zero-init network just never gets there, because it can't break the symmetry between its two hidden units.

The fix. Random initialization breaks that symmetry: the two hidden units start out slightly different, so their gradients diverge and they begin to specialize – one ends up detecting one region of the input space, the other a different region, and combined through the output layer they produce the curved boundary in Section 12.4 that XOR needs. This is why neural networks are normally initialized with small random values (Xavier/He initialization, in practice) rather than zeros. Zero-initialization is fine, and actually useful for a clean demonstration, for a single-layer perceptron on linearly separable gates like AND, OR, and NOT (Experiment 1) – there's only one output unit there, so there's no symmetry to break in the first place.

13. Conclusion

An MLP was built and trained with the architecture 784 → Dense(128, ReLU) → Dense(64, ReLU) → Dense(10, Softmax) on Fashion-MNIST, reaching 88.51% test accuracy after 20 epochs. Running automated hyperparameter optimization via RandomizedSearchCV, with 5-fold cross-validation over a search space of eight hyperparameters, turned up a wider, longer-trained configuration (2 hidden layers of 256 neurons, Sigmoid activation, Adam at learning rate 0.001, 30 epochs) that pushed test accuracy to 89.10% – a real improvement, but a modest one, and it came at roughly 34% higher training cost. Both models' confusion matrices point to the same underlying limitation: the network consistently struggles to tell apart visually similar upper-body garments (Shirt, T-shirt/top, Pullover, and Coat). That's largely a side effect of flattening each image into a 784-length vector and throwing away all spatial structure – a convolutional architecture would probably handle this better, since it could actually learn spatial features like collars, sleeve shapes, and necklines instead of treating every pixel as an independent, unrelated feature. Overall, this experiment showed both sides of MLPs: they're a strong, simple baseline for image classification, but they clearly hit a ceiling when the task calls for fine-grained, shape-based visual distinctions.

14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.